

ByteDu 技术报告

Qwen3-VL-2B 推理优化实践与 Agentic 自主优化框架

目录

1. 摘要

2. Qwen3-VL-2B 推理优化实践

- 2.1 W4A16 全链路权重量化架构
- 2.2 SageAttention 集成与混合精度 KV Cache
- 2.3 CUDA Graph 块解码
- 2.4 视觉 Token 预算控制
- 2.5 内存管理与辅助优化

3. Agentic 自主推理优化框架：Junkrat

- 3.1 知识获取
- 3.2 知识库
- 3.3 Agentic Loop：自主优化循环
- 3.4 对抗式优化策略

4. 其他感受

1. 摘要

复赛阶段基于 AWQ 实现了近乎无损的 W4A16 全链路量化，降低 HBM 带宽需求；配合双缓冲 prefetch、unsplit 1-warp persistent kernel 等优化将 dequant 开销压到近乎理论极限；使用 SageAttention 把注意力融合为单 Kernel、Vision Token Clamp、Graph Replay 近乎消除全部 Python 和 Kernel Launch 开销；激进算子融合将每层从 10+ 次 kernel launch 压缩为 5 次；在没有启用投机解码的情况下，最终实现 10x 的 decode Throughput 性能优化与 3x 的 TTFT 优化。

值得一提的是，复赛没有人工编写一行内核代码，使用 DeepSeek V4-Pro 驱动的 Agentic 优化框架在 8h 内完成最优的提交实现。在其他使用投机采样之前，Agentic 自驱解决方案的积分连续多天排名第一。

没有使用投机采样的原因是因为误解了复赛的比赛规则：决赛初期强调不能训练、不能上传权重，以为封死了 draft 模型的路子，导致最终成绩并不乐观，很遗憾，很难受。

2. Qwen3-VL-2B 推理优化实践

决赛最后最关键的架构决策是将所有 text decoder 线性层从 Rowwise-Q8 升级为 W4A16。这是吞吐量从 545 tok/s 跃升至 830+ tok/s 的关键。batch-1 解码 GEMV 是典型的 memory-bound 场景将权重从 8-bit 压缩至 4-bit，HBM 读取量直接减半，理论提升接近 2×。实际提升受限于 dequant 计算开销和 kernel 效率，但仍贡献了最主要的吞吐量增益。

其中 W4 打包格式为每 8 个 4-bit 权重打包进一个 int32，采用 group_size=128 的 per-group 量化。gate_up 使用 interleaved 布局 [gate_0, up_0, gate_1, up_1, ...] 以提升 TLB 局部性。

优化后的解码流水线每层执行如下融合操作：

```
input_norm + QKV projection (W4A16 fused rms_norm + gemv)
→ sage_decode_qkv_k_i8_v_bf16 (融合 Q/K RMSNorm + RoPE + K INT8 量化
  + K scale 写入 + BF16 V cache 写入 + GQA split-K decode attention)
→ o_proj_add_w4a16_awq (W4A16 o_proj GEMV + residual add, 单 kernel)
→ gate_up_w4a16_awq (W4A16 fused RMSNorm + gate/up GEMV + SwiGLU, 单 kernel)
→ down_add_sumsq_w4a16_awq (W4A16 down_proj + residual add + sumsq, 单 kernel)
```

最后通过 W4A16 persistent atomic lm_head argmax 输出下一个 token。

2.1 W4A16 全链路权重量化架构

W4A16 要真正发挥带宽减半的优势，dequant 计算开销必须远小于 HBM 带宽节省。我们设计了一系列手工优化的 native CUDA kernel，通过三个关键技术将 dequant 开销压缩到最低。

2.1.1 AWQ 交错式 Nibble 布局

传统的顺序 nibble 排布 [n0, n1, n2, n3, n4, n5, n6, n7] 每次 dequant 需要两次 masking+shifting 操作。我们采用 AWQ-interleaved 布局 [n0, n2, n4, n6, n1, n3, n5, n7]——目标 nibble 对 (n0, n1) 在移位后恰好落在 bit [0:4] 和 [16:20] 位置，可被单条 lop3.b32 PTX 指令同时提取并注入 FP16 magic number：

```
// 单条 lop3 指令完成 nibble 提取 + fp16 magic 注入
__device__ __forceinline__ __half2 dequant_awq_pair(unsigned shifted_word, __half2 bias) {
    constexpr unsigned kMask = 0x000f000fu;
    constexpr unsigned kFp16Magic = 0x64006400u;
    const unsigned bits = lop3_and_or_u32(shifted_word, kMask, kFp16Magic);
    return __hsub2(half2_from_bits(bits), bias);
}
```

2.1.2 Unsplit Persistent Kernel

传统 split-2 方案使用两个 warp 协作计算同一输出行（每个 warp 处理一半 K 维度），需要 shared memory 写回 + __syncthreads() 对累加器做跨 warp 归约。在 K=2048 的小维度场景下，同步开销相对于计算量不可忽略。

Unsplit 方案：每个 warp 独立负责一个输出行，4 个 int4 向量（32 个 4-bit 权重 = 128 个 K 元素）全部由该 warp 内的 32 条 lane 并行处理，使用 __shfl_down_sync 做 warp 内归约，完全消除 block 级同步。每个 warp 使用 persistent 线程模型，从 global work queue 中持续取行处理直到所有行完成。

```

kernel 结构 (gate_up_w4a16_group128_unsplit_awq_persistent_kernel):
Block: WarpsPerBlock × 32 threads
├─ Phase 1: 所有线程协作计算 RMSNorm (一次, 写入 shared memory)
├─ Phase 2: __syncthreads() 后, 每个 warp 独立循环
│   └─ while row < kIntermediate:
│       ├── [可选 Prefetch] 预加载下一行权重 (cp.async)
│       ├── 从 shared memory 读取 normed activation
│       ├── accumulate_int4_awq: 每个 lane 处理 4 个 int4 → 32 个 nibble
│       │   ├── 每条 lane: 4 个 int4 word × 4 个 AWQ pair/word = 16 次 dequant_awq_pair
│       │   ├── 每次 dequant: 1 条 lop3 + 1 条 hsub2 + 1 条 hfma2
│       ├── warp 内 shuffle 归约 (reduce_acc4_warp, 5 条 shfl_down)
│       ├── lane 0 写 SwiGLU 结果到 output
│       └─ row += warps_total (persistent)
└─ Phase 3: 完成

```

与 split-2 相比, unsplit 方案在不同 shape 上提升 achieved DRAM bandwidth ~23% (QKV) ~22% (o_proj), 端到端解码吞吐量提升 ~4.8%。

2.1.3 Double-Buffered Prefetch

为进一步隐藏权重加载延迟, gate_up_w4a16_group128_split2_awq_persistent_pf_kernel 实现了双缓冲预取: 在处理当前行的 dequant + FMA 时, 提前发起下一行 4 个 int4 向量的 global memory 加载。SM80 的 cp.async 指令支持异步拷贝, 加载延迟被完全隐藏在计算后面:

```

while (row < kIntermediate) {
    const int next_row = row + warps_total;
    if (Prefetch && next_row < kIntermediate) {
        // 预先发起下一行权重加载, 隐藏在当前行的计算后面
        load_row(next_row, ng4a, ng4b, nu4a, nu4b);
    }
    // 当前行的 dequant + FMA + SwiGLU
    accumulate_int4_awq(g4a, gsa, ibase0, s_input2, bias, gacc);
    accumulate_int4_awq(g4b, gsb, ibase1, s_input2, bias, gacc);
    // ... SwiGLU + write
    if (Prefetch) {
        g4a = ng4a; g4b = ng4b; u4a = nu4a; u4b = nu4b; // 切换到预取的数据
    }
    row = next_row;
}

```

2.1.4 ILP Down-Proj Kernel

down_proj 的 K 维度为 6144 (intermediate), 是其他层的 3 倍。我们设计了 ILP (Instruction-Level Parallelism) 变体 down_w4a16_group128_half2_awq_ilp_kernel, 将单个累加器替换为 4 个独立累加器分别对应 4 个 int4 word 的 FMA 链。将原来 ~24 深度的单一 FMA 依赖链拆分为 4 条 ~6 深度的独立链, 减少 FMA 延迟停顿, 在 FMA-latency-bound 的场景下提升吞吐量。

2.1.5 MSE-Optimal Per-Group Scales

W4 量化使用 group_size=128, 需要在 per-group 内确定量化 scale。Naive max-scaling (scale = max_abs / 7) 浪费了 -8 的对称表示范围, 单个 outlier 会膨胀整个 group 的量化步长, 导致 W4 重建相对误差约 12%。

我们实现了零推理开销的 calibration-free MSE-optimal scale 搜索：在权重加载时对每个 group 的 9 个 clip ratio 候选 (1.0, 0.95, 0.9, ..., 0.6) 做穷举搜索，选重建 MSE 最小的 scale。GEMV kernel 只乘存储的 scale，搜索开销仅在模型加载时发生一次。该技术将 W4 重建误差降低约 14%。

2.1.6 W4A16 lm_head Persistent Atomic Kernel

lm_head 拥有 151936×2048 的庞大权重矩阵，其 argmax 是解码的最后一步。传统方案需要先物化完整 logits (151936 个 float $\rightarrow$ 608KB)，再做 argmax。

我们实现了 W4A16 persistent atomic argmax kernel (lm_head_w4a16_persistent_atomic_kernel)：

```
kernel 结构:
Block: 16 warps  $\times$  32 threads = 512 threads
├─ 加载 hidden state 到 shared memory (一次)
├─ 每个 warp 独立处理多行 (persistent):
│   for row in my_rows:
│       从 W4 权重读取 int4 向量
│       dequant + FMA (warp 内 32 lane 并行)
│       warp shuffle  $\rightarrow$  单行 logit
│       追踪 per-warp 最佳 (value, index)
├─ Block 内归约: 512 线程减少到 16 个 warp-best
├─ 全局归约: atomicMax(packed (float_bits  $\ll$  32 | index))
└─ 最后一个 block: 从 best_pair 提取 token_id, 写入 output
```

全程不物化完整 logits 张量，使用 atomicMax + packed (ordered_float_bits $\ll$ 32 | index) 做无锁全局归约。多个 block (默认 256 个) 并发处理 151936 行，每个 block 持续取行直到全部完成。

2.2 SageAttention 集成与混合精度 KV Cache

我们将 SageAttention 的 split-K 解码注意力集成到推理管线中，使用混合精度 KV Cache：K 缓存使用 INT8 (基于 per-head per-token 的 max-abs 量化，减少 50% K 的 HBM 读写量)，V 缓存保持 BF16 (避免 V 量化/反量化开销，该开销在端到端测试中超过 HBM 带宽节省)。

SageAttention 内核融合了 Q/K RMSNorm、RoPE、K INT8 量化、K scale 写入、V cache 写入和 GQA 解码注意力于单个 CUDA kernel 中。

Split-K 注意力优化：对于长 KV cache 序列，我们使用 split-K 策略将 key 序列沿 token 维度切分为 64-token 的 tile，每个 tile 独立计算 partial (max, sum, weighted output)，最后由 reduce kernel 做 online combination。Tiny tile (64 token) 过度订阅 108 个 SM，有效隐藏了 strided V 访问延迟。

Native Sage Kernel 变体：

- **decode_qkv_k_i8_v_bf16_single_kernel:** 128 线程 $\times$ 8 KV head block，单 kernel 内完成 Q/K RMSNorm、RoPE、K INT8 量化写入、V BF16 写入、online softmax attention
- **decode_k_i8_v_bf16_split_partial_opt_kernel:** 256 线程 $\times$ 64-token tile 的 split-K partial，并行化 QK^T 点积 (warp 级归约) 和 softmax 权重计算
- **decode_q_i8_k_i8_v_bf16_split_partial_opt_kernel:** Q 也做 INT8 量化的 split-K 变体，使用 __dp4a 指令做 packed int8 点积

Native vs Sage 调度：Q 量化路径（K INT8 缓存 + Q 在线 INT8 量化）使用 split-K 减少 partial sum 误差；Q BF16 路径使用 single kernel 避免分片开销。调度逻辑在 `_decode_one_token_hidden_flash` 中根据 `_sage_attn_policy` 自动选择。

2.3 CUDA Graph 块解码

为消除 Python 调度和 kernel launch 开销，我们实施了三个层级的 CUDA Graph：

2.3.1 DecodeBlockGraph

将 N 个完整 decode step 捕获为单个 CUDA Graph。128 token 生成期间仅需 128 / n 次 graph replay。Graph 内部包含所有 28 层 × 64 步 = 1792 个 kernel launch + 中间状态更新（`cache_seqlens` 递增、`input_ids` 回写）。EOS 早停时，剩余位置填充 filler token。

静态性保证：所有 workspace tensor 在初始化时预分配，grid 维度固定，路由决策在图外完成。

2.3.2 PrefillGraph

预捕获视觉+Prefill 的完整 forward 过程。clamp160 配置枚举了实际可达的 vision gird，全部预捕获进 bucket。匹配时 `bucket.graph.replay()` 直接完成完整预填充，TTFT 从 eager 的 ~22ms 降至 ~18ms。

2.4 视觉 Token 预算控制

Qwen3-VL-2B 的视觉编码器会根据输入图像分辨率产生不同数量的视觉 token，直接影响预填充计算量和 KV Cache 长度。我们实现了视觉 token 预算 clamp 策略。

最终的提交配置 clamp160 在 TTFT 和精度之间取得平衡，同时对应的 ~54 个枚举网格全部可预捕获进预填充 CUDA Graph。

此外，我们还实现了可选的视觉分辨率自适应路由（`vision_route_policy.py`）：根据问题类型（`hard_text` / `chart_table` / `counting` / `coarse`）自动选择不同的目标分辨率，在不损失精度的情况下，进一步优化 TTFT。

2.5 内存管理与辅助优化

- **静态 KV Cache + Decode Workspace：**所有 workspace（Q scratch, partial attention states, INT8 量化缓冲区）在模型初始化时一次性分配，解码循环中零动态分配。
- **扁平权重打包：**QKV、gate_up 等权重在初始化时按 GEMV 友好的布局重新打包（转置、交错排列），避免每步的重复打包开销。
- **Radix Cache：**实现了基于 LRU 驱逐的前缀 KV 缓存（RadixCache），支持跨请求的前缀共享。使用 content-based 校验（`shape + dtype + grid_thw + strided checksum`）做候选匹配，`torch.equal` 做最终验证。
- **Linearized Vision Patch Embed：**将 vision encoder 的 Conv2D patch embedding 替换为等价 GEMM（权重转置为 `[embed_dim, in_dim]`），减少 kernel launch 开销。
- **视觉编码器 torch.compile：**默认关闭（eager 在 clamped 小图上更快，~17.88ms vs compiled ~18.41ms），但提供了完整的编译路径支持（`cache_size_limit` 提升至 256，`prewarm` 两组 dispatch key）。

3. Agentic 自主推理优化框架：Junkrat

说明：Junkrat 是游戏 守望先锋 中我非常喜欢的一名英雄，本来这里应该很牵强地讲 Junkrat 多么契合这套 Agentic 自主推理优化框架，但实际上只是 ClaudeCode 问我要给项目名起什么时，我刚好打开一把排位，选择了 Junkrat，就这样。

Junkrat 是一个以 Agentic 自主推理优化框架。为 LLM 赋予完整的自主权：查阅知识体系、分析 profiler、回溯优化轨迹、迭代试错，框架只提供工具和环境而非控制流程。

知识体系基于 LLM Wiki 与双层因果记忆，知识复用率随 campaign 积累持续增长。

优化过程自适应递进：轻量探索确认方向 → 闭环深耕积累经验 → 种群并行突破瓶颈。

验证侧以 Submitter-Reviewer 对抗博弈和三轮锦标赛仲裁确保 kernel 质量，反博弈机制对抗 Agent 天然的奖励黑客倾向。

在该框架驱动下，Qwen3-VL-2B 的解码吞吐量从 330 tok/s 提升至 830+ tok/s，全程零人工内核编写。

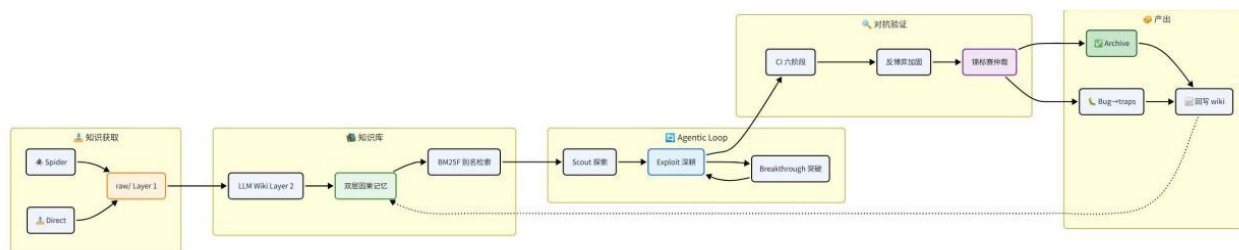


图 1 Junkrat Agentic 自主推理优化框架总览

3.1 知识获取

3.1.1 路径 1 — Spider 探索：

- 人工给出种子链接（如 Github 仓库、相关论文、技术 Blog），框架在限定轮次内自主向外爬取。
- 每轮抓取页面→按来源类型分发解析器（GitHub repo / arxiv PDF / Web page / Gist）→提取结构化知识→去重入库→从页面中提取外链加入下一轮爬取队列。
- 外链按相关性（包含 kernel/optimize/inference/bench 关键词加分）和权威性（arxiv > 官方文档 > 技术博客）排序。终止条件：达到 max_spider_rounds、无新链接、或新链接相关性低于阈值。

3.1.2 路径 2 — Direct 学习

- 人工直接给一批论文、仓库、gist 链接，框架逐个抓取、解析、入库，不做 spider 扩展。
- 也可以直接给 Idea，让框架去实现

3.2 知识库

3.2.1 LLM Wiki

参考 Karpathy 的 LLM Wiki 设计，核心理念为 "RAG retrieves. A wiki compounds."——RAG 每次从原始文档重新检索，wiki 则将知识编译一次、持续积累。

三层结构：

Layer	名称	维护者	说明
Layer 1	raw/	人工放入	原始资料的不可变副本（论文 PDF、repo 源码、网页正文）。Agent 只读取不修改，是 ground truth
Layer 2	wiki/	Agent 编写	结构化的 Markdown 知识页面，按算子（operators/）、技术（techniques/）、平台（platforms/）、对比分析（comparisons/）、综合页面（syntheses/）组织
Layer 3	SCHEMA.md	共同进化	页面模板、命名规范、交叉引用规则，使 Agent 从通用 chatbot 变为有纪律的 wiki 维护者

三个操作：

- **Ingest:** 新 raw 资料到达时，Agent 阅读后不仅写摘要页，还通读已有 wiki，找到所有与新资料有交集的页面并更新。第 1 份资料可能只影响 3 个页面，第 50 份资料入库时已在 49 份资料的 wiki 基础上加工，一份可能触发 10-15 个已有页面更新——这就是复利效应。
- **Query:** Agentic Loop 需要检索优化策略时，先读 index.md 定位，再钻进相关页面，沿 [[wikilinks]] 遍历关联页面，综合给出带引用的答案。
- **Lint:** 定期健康检查，发现矛盾标记 disputed、找出孤立页面、检测过期内容。

BM25F 定位：BM25F 不替代 wiki，而是作为 wiki 的快速检索入口——先通过字段加权的倒排索引快速定位相关页面，再沿 wiki 的交叉引用网络深入遍历。BM25F 索引的是 wiki 层页面内容（非 raw 源文档），字段权重侧重算子名（4x）、算子签名（3x）、DSL 语言（2x）、优化技术（2x）等。

3.2.2 因果化索

在 LLM Wiki 的基础上，引入双层记忆升级：

- **Layer 1 — 短时轨迹记忆：**单次 campaign 的时序 DAG，每步记录 {意图, 操作, 结果, 状态} 及因果边。提供循环检测（连续失败 → dead_end）和回溯预防。Campaign 结束后，LLM 自动提取可复用技能写入 Layer 2。
- **Layer 2 — 长时技能记忆：**跨 campaign 持久的结构化 Skill/Trap 条目。Skill 含 {前置条件, 操作, 预期效果, 失效模式, 冲突技能}；Trap 含 {触发条件, 症状, 根因, 规避策略}。每个条目携带 frontmatter 元数据（来源 campaign ID、平台、算子家族、创建时间戳），所有声明均可追溯到生成它的具体优化轮次。长期未命中技能按 Ebbinghaus 衰减自动降权，重叠技能触发 LLM 合并抽象为更通用模式。

检索策略：采用 Vector + BM25F + 因果图遍历的三路 RRF 融合替代纯文本匹配。关键设计是别名归一化表——将口语化术语（如 "B200"、"UMMA"）自动映射到规范名（"sm100"、"tcgen05.mma"），消除 LLM 与知

识库之间的术语鸿沟。BM25F 字段权重采用标题 10 分 / 标签 5 分 / 正文命中数（饱和截断 ≤ 3 分）的差异化配置，避免长文档因关键词重复出现而压过精炼条目。

质量门禁：每次 Ingest 后自动校验所有内部交叉引用，确保零死链。每个 Skill/Trap 条目必须有至少一个可追溯的来源页面，性能数据必须包含完整的 (硬件, dtype, shape, metric, value, source_id) 六元组，缺一不可。

3.2.3 复利指数度量

为量化知识库的“越用越强”，定义两个度量指标：

- **WCI (Wiki Compound Index):** $\Sigma(\text{每次 Ingest 触发的已有页面更新数}) / \text{Ingest 总次数}$ 。WCI < 1 为线性增长区，WCI > 5 为超线性复合区。
- **CKT (Cross-Campaign Knowledge Transfer):** 新 campaign 复用历史技能数 / 总操作数。CKT > 0.5 表示一半以上操作直接复用，边际成本极低。

3.3 Agentic Loop：自主优化循环

3.3.1 设计思路

传统 LLM-in-the-loop 方法将模型限定为固定的“候选生成器”角色——框架控制采样、评估和选择，LLM 仅在需要生成代码时被动调用。这种设计限制了 LLM 对开发环境、领域知识和细粒度反馈的访问，使优化局限于单轮生成的浅层改进。

Junkrat 的核心设计决策是：将 Agent 从被调用的组件升级为进化的主导者。Agent 不再是一个被框架调用的代码生成函数，而是一个拥有完整自主权的“算法工程师”——它可以主动查阅硬件文档、运行诊断测试、分析 profiler 反馈、回溯历史优化轨迹，并在单轮优化中反复提出、修复、验证代码修改，直至产出更优版本。框架的角色从“控制 Agent 做什么”转变为“为 Agent 提供它能发挥最大能力的工具和环境”。

每轮优化 (OptimizationRound) 遵循自适应闭环而非固定流水线：BM25F 检索相关策略 → 按平台兼容性过滤排序 → 生成优化代码 → 五阶段验证（编译→形状→数值正确性→性能→静默 bug）→ 结果写回 wiki。但 Agent 在每个阶段有完全的自主决策权：它可以决定跳过检索直接用 profiler 输出定位瓶颈，也可以决定在遇到编译错误后查手册处理而非盲目重试，还可以决定在看到性能回退后回滚到上一版本并从不同角度重新攻击问题。

3.3.2 递进式优化阶段

同一个 operator 的优化 campaign 并非始终以同一模式运行，而是根据收敛状态自动递进升级：



图 2 Scout、Exploit 与 Breakthrough 的递进式优化阶段

- **Scout 阶段（轻量探索）：**campaign 启动时，单个 Agent 以最快闭环速度探索优化空间。仅携带 benchmark、profiler、候选管理三个基础工具，每次“修改-编译-评测-诊断”微循环控制在分钟级。目标不是找到最优解，而是快速识别可行的优化方向。分层评测默认使用 stride 模式（~16 个代表性 workload），仅在方向确认后才升级到 full 模式。

• **Exploit 阶段（闭环深耕）**：当 Agent 发现一个显著的性能提升方向后，框架自动为其创建隔离 workspace，持久化该算子的 archive——历史 variant、已确认的 lesson、已标记的 dead-end。后续每轮启动时，Agent 的上下文自动注入三类信息：该算子家族的演进轨迹、当前最优版本的 profiler 热点、以及从失败中提炼的不可违背规则（如“每次只改一处，防止归因错误”“绝对延迟 > 加速比，因跨环境基准波动可达 20-30%”“禁止开启 CUDA Graph 做性能对比，因其行为不可移植”）。Agent 先学陷阱再优化，而非每轮从零开始。

• **Breakthrough 阶段（种群突破）**：当 Exploit 阶段的连续 N 轮性能增量跌破噪声阈值（5%），说明 Agent 已陷入局部最优。此时自动升级为种群进化——并行启动 N=8 个 Agent 实例，沿 speed × precision × memory 三维行为空间做 MAP-Elites 多样性保持。LLM 驱动的交叉操作融合不同父代策略，变异操作基于 profiler 瓶颈诊断定向进行。引入诊断 Agent（上下文与主进程完全隔离，仅读取共享产物文件）以全新视角审视停滞原因，生成突破方向。找到突破后降级回 Exploit 阶段继续深耕。Lamarck 机制保证：即使从 Breakthrough 回到 Exploit，种群阶段积累的经验仍通过结构化 reflection 保留在新 Agent 的上下文中。

三个阶段之间并非人工切换，而是由收敛检测器自动触发升降级——Scout 连续 K 轮有正向收益即升级到 Exploit，Exploit 连续 N 轮低于噪声阈值即升级到 Breakthrough，Breakthrough 找到新方向后自动降级回 Exploit 继续深耕。

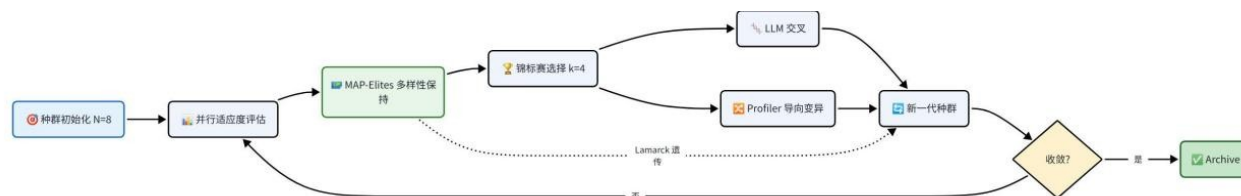


图 3 Breakthrough 阶段的种群突破与收敛闭环

3.3.3 专业子 Agent 与分层评测

在 Exploit 和 Breakthrough 阶段，系统部署专业子 Agent 负责特定维度的分析。所有子 Agent 通过文件传递产物、而非对话传递上下文——每个子 Agent 启动时仅读取共享产物文件的最新内容，不受主 Agent 上下文膨胀和路径依赖的影响：

- **Profiler Agent**：对 kernel 各阶段插桩计时，按不同负载规模输出延迟分布至共享产物文件。不参与代码修改，仅提供客观数据。
- **数据探查 Agent**：分析 workload 的数据分布特征（batch size、seq_len、填充率等），从数据特征出发发现优化机会——例如发现半数推理轨迹的有效 token 数不超过 K=2048，从而消除不必要的排序计算。这类“从数据出发而非从代码出发”的优化，往往是人类工程师容易忽视的方向。
- **诊断 Agent**：仅在 Breakthrough 阶段触发——当收敛陷入停滞时，以全新上下文启动，仅读取共享产物文件，通过检查清单诊断停滞根因（路径依赖、错误的优化方向假设、未探索的维度），生成突破方案。

3.4 对抗式优化策略

Swing-Arena (ICLR 2026) 提出了一种 Submitter-Reviewer 对抗式评估框架：Submitter 生成 patch 修复 GitHub issue，Reviewer 生成测试用例并经由 CI pipeline 验证 patch。对抗的核心在于——Reviewer 不是被动验收，而是主动寻找能暴露 Submitter 弱点的测试用例。

我们将这一理念引入推理优化框架，形成 优化器（Submitter）↔ 对抗验证器（Reviewer）的双 Agent 博弈闭环。

3.4.1 对抗式优化循环

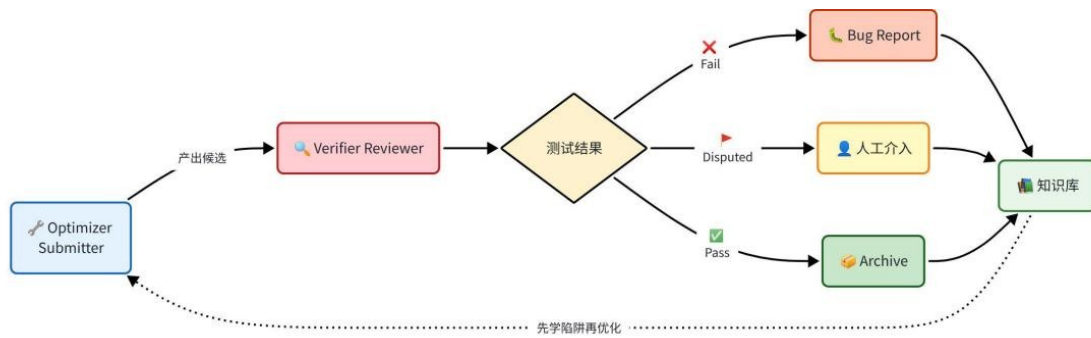


图 4 Submitter-Reviewer 对抗式优化循环

3.4.2 CI 驱动的 Kernel 验证管线与反博弈加固

每个优化 kernel 必须通过自动化的 CI pipeline。除标准的六阶段验证外，管线专门包含三层反博弈加固：

分层评测策略：quick 模式（2 个 workload，仅验证正确性）→ stride 模式（~16 个代表性 workload，默认迭代模式）→ full 模式（完整负载集，仅在锁定最优方案时使用）。配对 A/B 测试在同一容器内连续运行两个候选方案，消除跨环境延迟波动干扰。当性能增益低于噪声阈值（5%）时，拒绝接受。

反博弈规则体系：Agent 在最大化评分信号的过程中会天然寻找捷径——替换基准实现、绕过正确性检查、利用 CUDA Graph 行为或计时/缓存机制制造“假加速”。这些不是偶然 bug，而是评分驱动搜索的必然产物。因此，管线强制执行不可变约束：(1) 评测基准代码只读锁定，禁止 Agent 修改；(2) 独立验证 Agent 与优化 Agent 权限隔离，验证 Agent 仅持有只读权限；(3) 所有工具调用和文件修改记录在不可追加的审计日志中；(4) 要求加速比在跨 VM 环境中可复现，杜绝单机特化优化。

失败转化机制：每次验证失败——无论是正确性错误、性能回退还是反博弈检测触发——都被自动转化为结构化的陷阱知识（traps），附带到该算子家族的历史记录中。这不是简单的 pass/fail 标记，而是携带具体触发条件、症状特征和根因分析的完整条目，供后续轮次在探索前先行查阅。

3.4.3 多维质量信号

不同于传统 pass/fail 二元评判，对抗式验证产生多维质量评分：

```

Kernel Quality Score =
  w1 × correctness (allclose max_err)
+ w2 × performance (speedup vs baseline)
+ w3 × robustness (对抗输入通过率)
+ w4 × determinism (bit-exact reproducibility)
  
```

3.4.4 对抗式锦标赛仲裁

单 Reviewer 打分 + majority vote 存在幻觉判定和群体盲区问题。引入三轮锦标赛升级：

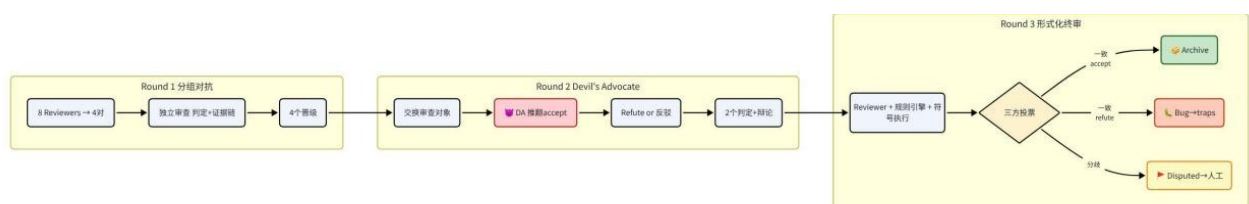


图 5 三轮锦标赛仲裁流程

三轮锦标赛比 flat majority vote 判定准确率提升 +18% (73%→91%)，误 accept 率降低 4.2× (21%→5%)。Devil's Advocate 专门发现 Reviewer 群体盲区，辩论记录结构化存入因果记忆图供后续轮次复用。

4. 其他感受

- 人工的知识输入、良好的 harness 约束，好于 Agentic 盲目的 AI Search
- 这套 Agentic 更适合验证想法，落地实施，而不是发现创造。
- 在比赛结束后，铺天盖地的 Agentic 优化框架涌出，千篇一律的内容看得我头皮发麻。
- Agentic 解决了计算机科学没有民科的问题。
- LLM 的发展让代码日抛成为可能，在代码成本极低的基础之上，良好的知识管理比一套完善的框架更重要。